C++ Programming Practice Questions

Introduction

This document contains practice questions for each topic covered in the C++ Programming Examples book. Each section includes 5 scenario-based questions that require you to "Write a program" to solve real-world problems. These questions are designed to test your understanding and application of the concepts.

Chapter 1: Basic C++ Concepts

1.1 Inline Functions

1. Write a program that simulates a temperature monitoring system where inline functions are used to convert Celsius to Fahrenheit and check if the temperature exceeds a threshold, displaying alerts for a series of temperature readings.
2. Write a program for a simple calculator that uses inline functions for basic arithmetic operations (addition, subtraction, multiplication, division) and handles user input for two numbers and an operation choice.
3. Write a program that manages employee salaries where inline functions calculate annual bonuses based on performance ratings and display the total compensation for multiple employees.
4. Write a program for a library system that uses inline functions to calculate overdue fines based on days late and book type, processing a list of borrowed books.
5. Write a program that simulates a vending machine where inline functions validate coin inputs, calculate change, and dispense items based on user selections.

1.2 Reference Variables

1. Write a program that manages a student database where reference variables are used to update student grades and calculate GPA for a class of students.
2. Write a program for a banking system that uses reference parameters to transfer money between accounts, updating balances and transaction histories.
3. Write a program that sorts an array of employee records using reference variables in a swap function, displaying the sorted list by salary.
4. Write a program for a game inventory system where reference variables modify item quantities and calculate total value when buying or selling items.
5. Write a program that processes survey responses using reference variables to count votes for different options and display the results with percentages.

1.3 Static Variables

1. Write a program that tracks website visitors using a static counter in a function that logs each visit, displaying the total visitor count across multiple page accesses.
2. Write a program for a library that uses static variables to keep track of total books borrowed and available, updating counts when books are checked out or returned.
3. Write a program that simulates a coffee shop where static variables track daily sales and customer count, generating a daily report at the end of the day.
4. Write a program for an online quiz system that uses static variables to maintain the current question number and score, displaying progress after each answer.
5. Write a program that manages parking spaces using static variables to count occupied and available spots, updating the display as cars enter and leave.

1.4 External Variables

1. Write a program for a company that uses global variables to store company-wide settings like tax rate and currency, calculating employee salaries and displaying payslips.
2. Write a program that manages a shared shopping cart across multiple functions using global variables, adding items, calculating totals, and applying discounts.
3. Write a program for a weather station that uses global variables to store current temperature, humidity, and pressure, updating readings from sensors and displaying forecasts.
4. Write a program that coordinates multiple threads in a factory simulation using global variables for production counts and quality checks, generating reports.
5. Write a program for a music player that uses global variables to maintain playlist information and current track, allowing play, pause, next, and previous operations.

1.5 Scope Resolution Operator

1. Write a program for a multi-level company hierarchy where scope resolution operator accesses global company policies while local department rules override them for specific calculations.
2. Write a program that manages different namespaces for mathematical constants and uses scope resolution to access PI and E in calculations for various geometric shapes.
3. Write a program for a game with multiple levels where scope resolution distinguishes between global game settings and level-specific configurations.
4. Write a program that handles configuration files with global settings overridden by user preferences, using scope resolution to access the correct values.
5. Write a program for a scientific calculator that uses scope resolution to access mathematical functions from different libraries while maintaining local variables.

1.6 Preprocessors

1. Write a program that uses preprocessor directives to create a configurable logging system with different log levels (DEBUG, INFO, ERROR) based on compilation flags.

2. Write a program for a cross-platform application that uses conditional compilation to handle different operating systems (Windows, Linux, macOS) for file path operations.
3. Write a program that defines macros for unit conversions (inches to cm, pounds to kg) and uses them in a measurement calculator for various recipes.
4. Write a program that uses include guards in header files to prevent multiple inclusions, demonstrating with a simple class hierarchy.
5. Write a program that uses preprocessor macros to implement a simple assert function for debugging mathematical operations in a calculator.

Chapter 2: Arrays and Pointers

2.1 Pointer Basics

1. Write a program that dynamically manages a list of student names using pointers, allowing addition, deletion, and display of names in a classroom management system.
2. Write a program for an image processing application that uses pointers to manipulate pixel values in a grayscale image array.
3. Write a program that implements a simple text editor using pointers to navigate through a character buffer for cursor movement and text insertion.
4. Write a program for a contact list that uses pointers to store and search through phone numbers and names efficiently.
5. Write a program that simulates memory allocation in an operating system using pointers to manage free and allocated memory blocks.

2.2 Pointer Arithmetic

1. Write a program that implements string reversal using pointer arithmetic, processing multiple input strings from a file.
2. Write a program for a matrix calculator that uses pointer arithmetic to perform addition and multiplication of two-dimensional arrays.
3. Write a program that searches for patterns in a text using pointer arithmetic to implement a simple string matching algorithm.
4. Write a program for audio processing that uses pointer arithmetic to apply filters to audio samples stored in an array.
5. Write a program that manages a circular buffer for network packets using pointer arithmetic to handle wrap-around efficiently.

2.3 Pointers to Functions

1. Write a program that implements a plugin system for image filters using function pointers, allowing users to apply different effects to images.
2. Write a program for a calculator that uses function pointers to implement different mathematical operations selected by the user.

3. Write a program that sorts an array using different sorting algorithms selected via function pointers (bubble sort, quick sort, merge sort).
4. Write a program for event handling in a GUI system where function pointers are used to register and call callback functions for button clicks.
5. Write a program that implements a state machine using function pointers to transition between different states based on events.

2.4 Returning Pointers

1. Write a program that implements a string tokenizer that returns pointers to substrings, parsing a sentence into words.
2. Write a program for a database system that returns pointers to records matching search criteria from a collection of employee data.
3. Write a program that implements a memory pool allocator returning pointers to allocated blocks for efficient memory management.
4. Write a program for text analysis that returns pointers to the most frequent words in a document.
5. Write a program that implements a simple cache system returning pointers to cached data or null if not found.

Chapter 3: Strings and Character Handling

3.1 String Length

1. Write a program that analyzes text files to find the longest word, using string length calculations to process multiple documents.
2. Write a program for password validation that checks length requirements and displays feedback for various password attempts.
3. Write a program that formats text by wrapping lines at a maximum length, using string length to determine line breaks.
4. Write a program for a messaging app that truncates messages exceeding character limits, showing previews with string length information.
5. Write a program that generates usernames from full names, using string length to create unique identifiers of appropriate size.

3.2 String Copy

1. Write a program that backs up configuration files by copying them to a backup directory with timestamped names.
2. Write a program for a document editor that implements copy-paste functionality, handling text selection and insertion.
3. Write a program that creates user profiles by copying template data and customizing it with user input.
4. Write a program for version control that creates snapshots by copying file contents to version directories.
5. Write a program that generates reports by copying data from multiple sources into a formatted output file.

3.3 String Concatenation

1. Write a program that builds SQL queries by concatenating user inputs with predefined templates for a database application.
2. Write a program for email composition that concatenates subject, body, and signature into complete email messages.
3. Write a program that generates HTML pages by concatenating tags and content for a simple website builder.
4. Write a program for log file analysis that concatenates date, time, and event information into comprehensive log entries.
5. Write a program that creates mailing labels by concatenating address components for a postal service application.

3.4 String Comparison

1. Write a program that sorts a list of names alphabetically for a contact management system.
2. Write a program for user authentication that compares entered passwords with stored hashes.
3. Write a program that finds duplicate files by comparing file names and contents in a directory.
4. Write a program for a quiz application that compares user answers with correct answers and calculates scores.
5. Write a program that merges two sorted lists of numbers by comparing elements during the merge process.

3.5 Character Input/Output

1. Write a program that implements a simple text-based game where user input controls character movement and actions.
2. Write a program for a typing tutor that reads user keystrokes and provides feedback on typing speed and accuracy.
3. Write a program that processes configuration files by reading key-value pairs and storing them in memory.
4. Write a program for a chat application that handles real-time message input and display with proper formatting.
5. Write a program that implements a command-line calculator accepting mathematical expressions as character input.

Chapter 4: Structures and Unions

4.1 Structure Basics

1. Write a program that manages a library catalog using structures to store book information and implement search functionality.
2. Write a program for employee management that uses structures to store personal and professional details, calculating salaries and benefits.
3. Write a program that tracks inventory in a warehouse using structures for product details and quantity management.
4. Write a program for student grade management that uses structures to store course information and calculate GPAs.
5. Write a program that manages a movie database using structures for film details, ratings, and user reviews.

4.2 Arrays of Structures

1. Write a program that manages a sports league using arrays of structures for teams, players, and match results.
2. Write a program for a hospital management system using arrays of structures for patients, doctors, and appointments.
3. Write a program that handles flight bookings using arrays of structures for flights, passengers, and seat assignments.
4. Write a program for a music library using arrays of structures for songs, artists, and playlists.
5. Write a program that manages a retail store inventory using arrays of structures for products, suppliers, and sales data.

4.3 Unions

1. Write a program that handles different data types in a calculator using unions for numeric values and operations.
2. Write a program for a communication protocol that uses unions to handle different message types (text, numbers, commands).
3. Write a program that manages device settings using unions for different configuration options (network, display, audio).
4. Write a program for a graphics system that uses unions to store different shape properties (circle radius, rectangle dimensions).
5. Write a program that handles database records using unions for different field types (integer, string, date).

Chapter 5: Classes and Objects

5.1 Class Fundamentals

1. Write a program that implements a BankAccount class with methods for deposit, withdrawal, and balance inquiry.
2. Write a program that creates a Car class with properties like make, model, year, and methods to start, stop, and drive.
3. Write a program that implements a Student class with methods to enroll in courses, calculate GPA, and display transcript.
4. Write a program that creates a Product class for an e-commerce system with pricing, inventory, and discount calculations.
5. Write a program that implements a Person class with contact information and methods for updating and displaying details.

5.2 Arrays of Objects

1. Write a program that manages a fleet of vehicles using an array of Car objects with maintenance scheduling.
2. Write a program for a bookstore that uses an array of Book objects to manage inventory and sales.
3. Write a program that handles employee payroll using an array of Employee objects with salary calculations.
4. Write a program for a music streaming service using an array of Song objects with playlist management.
5. Write a program that manages hotel reservations using an array of Room objects with booking and checkout.

5.3 Static Members

1. Write a program that implements a Counter class with static members to track instances across the application.
2. Write a program for a company that uses static members in an Employee class to track total employees and average salary.
3. Write a program that manages database connections using static members to maintain connection pools.
4. Write a program for a game that uses static members to track high scores and game statistics.
5. Write a program that implements a Logger class with static methods for different log levels and file output.

Chapter 6: Constructors and Destructors

6.1 Default Constructor

1. Write a program that creates a FileHandler class with a default constructor that opens a default log file.
2. Write a program that implements a Timer class with a default constructor that starts timing automatically.
3. Write a program that creates a DatabaseConnection class with a default constructor for local database access.
4. Write a program for a graphics application with a Window class that uses default constructor for standard window creation.
5. Write a program that manages network sockets with a Socket class using default constructor for localhost connection.

6.2 Parameterized Constructor

1. Write a program that creates a Rectangle class with parameterized constructor for width and height, calculating area and perimeter.
2. Write a program for a banking system with Account class parameterized constructor for initial balance and account type.
3. Write a program that implements a Date class with parameterized constructor for day, month, year validation.
4. Write a program for a graphics library with Color class parameterized constructor for RGB values.
5. Write a program that manages files with File class parameterized constructor for filename and access mode.

6.3 Copy Constructor

1. Write a program that implements a Matrix class with copy constructor for matrix duplication in mathematical operations.
2. Write a program for document management with Document class copy constructor for creating document versions.
3. Write a program that handles image processing with Image class copy constructor for creating image copies.
4. Write a program for a playlist system with Song class copy constructor for adding songs to multiple playlists.
5. Write a program that manages user profiles with Profile class copy constructor for account duplication.

6.4 Destructor

1. Write a program that implements a FileManager class with destructor that automatically closes open files.
2. Write a program for network communication with Connection class destructor that closes network connections.
3. Write a program that manages memory with Buffer class destructor that frees allocated memory.
4. Write a program for a game with Player class destructor that saves player progress on exit.
5. Write a program that handles database operations with Query class destructor that closes database cursors.

Chapter 7: Inheritance

7.1 Single Inheritance

1. Write a program that creates a hierarchy of shapes with Circle and Rectangle inheriting from Shape class.
2. Write a program for an employee management system with Manager and Developer classes inheriting from Employee.
3. Write a program that implements vehicle hierarchy with Car and Motorcycle inheriting from Vehicle class.
4. Write a program for a publication system with Book and Magazine classes inheriting from Publication.
5. Write a program that creates animal hierarchy with Dog and Cat classes inheriting from Animal.

7.2 Multiple Inheritance

1. Write a program that implements a FlyingCar class inheriting from both Car and Airplane classes.
2. Write a program for a smart device that inherits from both Phone and Computer classes.
3. Write a program that creates an AmphibiousVehicle class inheriting from both LandVehicle and WaterVehicle.
4. Write a program for a multimedia file that inherits from both Audio and Video classes.
5. Write a program that implements a TeachingAssistant class inheriting from both Teacher and Student.

7.3 Multilevel Inheritance

1. Write a program that creates a hierarchy of employees with CEO, Manager, and Developer in a chain.
2. Write a program for educational institutions with University, College, and Department in multilevel inheritance.
3. Write a program that implements transportation hierarchy with Vehicle, Car, and SportsCar.
4. Write a program for file systems with File, Document, and PDF in multilevel inheritance.
5. Write a program that creates biological classification with Animal, Mammal, and Human.

Chapter 8: Operator Overloading

8.1 Unary Operator Overloading

1. Write a program that implements a Counter class with overloaded ++ operator for incrementing values.
2. Write a program for complex numbers with overloaded - operator for negation.
3. Write a program that creates a Time class with overloaded ! operator for time validation.
4. Write a program for boolean operations with overloaded ~ operator for logical NOT.
5. Write a program that implements a Distance class with overloaded -- operator for decrementing.

8.2 Binary Operator Overloading

1. Write a program that implements Complex number class with overloaded + and - operators.
2. Write a program for matrix operations with overloaded * operator for matrix multiplication.
3. Write a program that creates a Fraction class with overloaded / operator for division.
4. Write a program for date calculations with overloaded - operator for date differences.
5. Write a program that implements String class with overloaded + operator for concatenation.

8.3 Friend Functions

1. Write a program that uses friend functions for overloading << and >> operators in a custom class.
2. Write a program for complex number operations using friend functions for operator overloading.
3. Write a program that implements Point class with friend functions for distance calculations.
4. Write a program for financial calculations with Money class using friend functions for operations.
5. Write a program that handles geometric shapes with friend functions for area comparisons.

Chapter 9: Dynamic Memory Management

9.1 malloc

1. Write a program that dynamically allocates memory for a 2D array using malloc and performs matrix operations.
2. Write a program for text processing that uses malloc to allocate memory for variable-length strings.
3. Write a program that implements a dynamic stack using malloc for memory allocation.
4. Write a program for image manipulation that uses malloc to allocate pixel buffers of varying sizes.
5. Write a program that manages a linked list with dynamic node allocation using malloc.

9.2 calloc

1. Write a program that uses calloc to initialize an array of structures for student records.
2. Write a program for numerical computing that uses calloc to create zero-initialized matrices.
3. Write a program that implements a hash table with calloc for initializing bucket arrays.
4. Write a program for audio processing that uses calloc to allocate and zero-initialize audio buffers.
5. Write a program that manages configuration settings using calloc for default value initialization.

Advanced Programming Exercises

1. Write a program that implements a complete library management system using classes, inheritance, and file I/O.
2. Write a program that creates a simple interpreter for mathematical expressions using dynamic memory and operator overloading.
3. Write a program that simulates an operating system process scheduler using data structures and algorithms.
4. Write a program that implements a text-based adventure game using object-oriented principles and file handling.
5. Write a program that creates a simple database system with CRUD operations using dynamic memory management.

6. Write a program that implements various sorting algorithms and compares their performance using timing functions.
7. Write a program that creates a graphics library with shape classes and rendering capabilities.
8. Write a program that implements a network packet analyzer using structures and pointer manipulation.
9. Write a program that creates a multi-threaded web crawler with proper synchronization.
10. Write a program that implements a compression algorithm using bitwise operations and dynamic memory.